

Week-6

Q1: Explain the roles of the **Driver**, **Cluster Manager**, and **Executor** in a Spark application.

Ans:-

A Spark application consists of three main components: the **Driver**, **Cluster Manager**, and **Executors**.

- **Driver:** The Driver is the main program that starts the Spark application. It creates the SparkSession, converts the user code into tasks, schedules them, and collects the final results from the executors.
- **Cluster Manager:** The Cluster Manager is responsible for managing the resources of the cluster. It allocates CPU cores and memory to the application and launches executors on worker nodes. Examples include Standalone, YARN, Mesos, and Kubernetes.
- **Executor:** Executors are worker processes that execute the tasks assigned by the Driver. They perform computations, store intermediate data, and return the results to the Driver.

Q2: How does Spark's **Lazy Evaluation** strategy improve performance when chain-processing large datasets?

Ans:-

Spark uses **Lazy Evaluation**, which means that transformations are not executed immediately. Instead, Spark records all the transformations and creates a logical execution plan called a **Directed Acyclic Graph (DAG)**. The computation starts only when an action such as `show()`, `count()`, or `collect()` is called.

This improves performance by combining multiple transformations into a single optimized execution plan, reducing unnecessary data movement, minimizing disk I/O, and eliminating redundant computations. As a result, Spark processes large datasets more efficiently.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
20/07/23 22:43:13: WARN NativeCodeLoader: Unable to load native Hadoop library for your platform... using duck
```

product_id	product_name	category	old_name	price	status	amount	base_price	user_id	region	priority
P101	Samsung TV	Electronics	TV	55000	Completed	55000	55000	U101	North	High
P102	HP Laptop	Electronics	Laptop	72000	Pending	72000	72000	U102	West	Medium
P103	Rice Bag	Groceries	Rice	2500	Completed	2500	2500	U103	South	Low
P104	Office Chair	Furniture	Chair	8500	Completed	8500	8500	U104	East	Medium
P105	Mixer Grinder	Electronics	Mixer	4200	Cancelled	4200	4200	U105	North	High
P106	Dining Table	Furniture	Table	25000	Completed	25000	25000	U106	West	Low
P107	Jeans	Clothing	Jeans	1800	Pending	1800	1800	U107	South	Medium
P108	Smartphone	Electronics	Phone	35000	Completed	35000	35000	NULL	North	High
P109	Milk	Groceries	Milk	60	Completed	60	60	U109	East	Low
P110	Air Conditioner	Electronics	AC	48000	Completed	48000	48000	U110	West	High
P111	Bookshelf	Furniture	Shelf	12000	Pending	12000	12000	U111	North	Medium
P112	T-Shirt	Clothing	Tee	900	Completed	900	900	U112	South	Low

Q4: What is the difference between **CSV** and **Parquet** in terms of storage (row-based vs. columnar) and why does it matter for performance?

Ans:-

CSV is a **row-based** file format where all values of a row are stored together. It is simple, human-readable, and widely supported, but it requires reading the entire file even if only a few columns are needed.

Parquet is a **columnar** file format where values of the same column are stored together. This allows Spark to read only the required columns, use better compression, and reduce disk I/O. As a result, Parquet provides faster query performance and lower storage requirements, making it the preferred format for big data applications.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

```

+-----+-----+
|product_id|price|
+-----+-----+
|      P101|55000|
|      P102|72000|
|      P105| 4200|
|      P108|35000|
|      P110|48000|
|      P113|  NULL|
|      P115| 3000|
|      P117|65000|
|      P119|  NULL|
|      P120| 1200|
+-----+-----+

```

Q6: Write the code to "revise" a DataFrame by renaming the column `old_name` to `new_name` and casting the price column from a String to a Double.

```

root
|-- product_id: string (nullable = true)
|-- product_name: string (nullable = true)
|-- category: string (nullable = true)
|-- new_name: string (nullable = true)
|-- price: double (nullable = true)
|-- status: string (nullable = true)
|-- amount: integer (nullable = true)
|-- base_price: integer (nullable = true)
|-- user_id: string (nullable = true)
|-- region: string (nullable = true)
|-- priority: string (nullable = true)

```

product_id	product_name	category	new_name	price	status	amount	base_price	user_id	region	priority
P101	Samsung TV	Electronics	TV	55000.0	Completed	55000	55000	U101	North	High
P102	HP Laptop	Electronics	Laptop	72000.0	Pending	72000	72000	U102	West	Medium
P103	Rice Bag	Groceries	Rice	2500.0	Completed	2500	2500	U103	South	Low
P104	Office Chair	Furniture	Chair	8500.0	Completed	8500	8500	U104	East	Medium
P105	Mixer Grinder	Electronics	Mixer	4200.0	Cancelled	4200	4200	U105	North	High
P106	Dining Table	Furniture	Table	25000.0	Completed	25000	25000	U106	West	Low
P107	Jeans	Clothing	Jeans	1800.0	Pending	1800	1800	U107	South	Medium
P108	Smartphone	Electronics	Phone	35000.0	Completed	35000	35000	NULL	North	High
P109	Milk	Groceries	Milk	60.0	Completed	60	60	U109	East	Low
P110	Air Conditioner	Electronics	AC	48000.0	Completed	48000	48000	U110	West	High
P111	Bookshelf	Furniture	Shelf	12000.0	Pending	12000	12000	U111	North	Medium
P112	T-Shirt	Clothing	Tee	900.0	Completed	900	900	U112	South	Low
P113	Washing Machine	Electronics	Washer	NULL	Completed	38000	38000	U113	North	High
P114	Sofa	Furniture	Sofa	45000.0	Cancelled	45000	45000	U114	West	High
P115	Headphones	Electronics	Headset	3000.0	Completed	3000	3000	U115	North	Medium
P116	Cooking Oil	Groceries	Oil	2200.0	Completed	2200	2200	U116	East	High
P117	Refrigerator	Electronics	Fridge	65000.0	Pending	65000	65000	U117	South	High
P118	Curtains	Home Decor	Curtain	1500.0	Completed	1500	1500	U118	North	Low
P119	Microwave Oven	Electronics	Microwave	NULL	Completed	11000	11000	U119	West	Medium
P120	Study Lamp	Electronics	Lamp	1200.0	Completed	1200	1200	U120	North	High

Q7: How does Spark use the **Lineage Graph (DAG)** to provide fault tolerance if a worker node fails?

Ans:-

Spark maintains a **Lineage Graph**, also known as a Directed Acyclic Graph (DAG), which records all the transformations applied to the data. If a worker node fails and some data is lost, Spark does not need to reload the entire dataset. Instead, it uses the lineage information to recompute only the lost partitions from the original data. This approach provides fault tolerance without creating multiple copies of the data, making Spark both reliable and efficient.

Q8: Write a query to filter a DataFrame `df_orders` for rows where the status is 'Completed' AND the amount is greater than 1000.

product_id	product_name	category	old_name	price	status	amount	base_price	user_id	region	priority
P101	Samsung TV	Electronics	TV	55000	Completed	55000	55000	U101	North	High
P103	Rice Bag	Groceries	Rice	2500	Completed	2500	2500	U103	South	Low
P104	Office Chair	Furniture	Chair	8500	Completed	8500	8500	U104	East	Medium
P106	Dining Table	Furniture	Table	25000	Completed	25000	25000	U106	West	Low
P108	Smartphone	Electronics	Phone	35000	Completed	35000	35000	NULL	North	High
P110	Air Conditioner	Electronics	AC	48000	Completed	48000	48000	U110	West	High
P113	Washing Machine	Electronics	Washer	NULL	Completed	38000	38000	U113	North	High
P115	Headphones	Electronics	Headset	3000	Completed	3000	3000	U115	North	Medium
P116	Cooking Oil	Groceries	Oil	2200	Completed	2200	2200	U116	East	High
P118	Curtains	Home Decor	Curtain	1500	Completed	1500	1500	U118	North	Low
P119	Microwave Oven	Electronics	Microwave	NULL	Completed	11000	11000	U119	West	Medium
P120	Study Lamp	Electronics	Lamp	1200	Completed	1200	1200	U120	North	High

Q9: Explain the concept of **Predicate Pushdown** in Parquet and how it affects the amount of data loaded into memory.

Ans:-

Predicate Pushdown is an optimization technique used by Parquet where filter conditions are applied while reading the file. Instead of loading all the data into memory, Spark reads only the rows that satisfy the filter condition. This reduces disk I/O, decreases memory usage, and improves query performance, especially when working with large datasets.

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

product_id	product_name	category	old_name	price	status	amount	base_price	user_id	region	priority	final_price
P101	Samsung TV	Electronics	TV	55000	Completed	55000	55000	U101	North	High	64900.0
P102	HP Laptop	Electronics	Laptop	72000	Pending	72000	72000	U102	West	Medium	84960.0
P103	Rice Bag	Groceries	Rice	2500	Completed	2500	2500	U103	South	Low	2950.0
P104	Office Chair	Furniture	Chair	8500	Completed	8500	8500	U104	East	Medium	10030.0
P105	Mixer Grinder	Electronics	Mixer	4200	Cancelled	4200	4200	U105	North	High	4956.0
P106	Dining Table	Furniture	Table	25000	Completed	25000	25000	U106	West	Low	29500.0
P107	Jeans	Clothing	Jeans	1800	Pending	1800	1800	U107	South	Medium	2124.0
P108	Smartphone	Electronics	Phone	35000	Completed	35000	35000	NULL	North	High	41300.0
P109	Milk	Groceries	Milk	60	Completed	60	60	U109	East	Low	70.8
P110	Air Conditioner	Electronics	AC	48000	Completed	48000	48000	U110	West	High	56640.0
P111	Bookshelf	Furniture	Shelf	12000	Pending	12000	12000	U111	North	Medium	14160.0
P112	T-Shirt	Clothing	Tee	900	Completed	900	900	U112	South	Low	1062.0
P113	Washing Machine	Electronics	Washer	NULL	Completed	38000	38000	U113	North	High	44840.0
P114	Sofa	Furniture	Sofa	45000	Cancelled	45000	45000	U114	West	High	53100.0
P115	Headphones	Electronics	Headset	3000	Completed	3000	3000	U115	North	Medium	3540.0
P116	Cooking Oil	Groceries	Oil	2200	Completed	2200	2200	U116	East	High	2596.0
P117	Refrigerator	Electronics	Fridge	65000	Pending	65000	65000	U117	South	High	76700.0
P118	Curtains	Home Decor	Curtain	1500	Completed	1500	1500	U118	North	Low	1770.0
P119	Microwave Oven	Electronics	Microwave	NULL	Completed	11000	11000	U119	West	Medium	12980.0
P120	Study Lamp	Electronics	Lamp	1200	Completed	1200	1200	U120	North	High	1416.0

Q11: What is the difference between **Transformations** and **Actions**? Provide two examples of each.

Ans:-

Transformations are operations that create a new DataFrame or RDD without immediately executing the computation. Spark records these operations and executes them only when an action is called.

Examples of Transformations:

- filter()
- select()

Actions are operations that trigger the execution of all pending transformations and produce a result or output.

Examples of Actions:

- show()
- collect()

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

```
[14] df.write.mode("overwrite").parquet("input_parquet")
✓ 0s

[8] parquet_df = spark.read.parquet("input_parquet")
✓ 1s
parquet_df.show()
```

product_id	product_name	category	old_name	price	status	amount	base_price	user_id	region	priority
P101	Samsung TV	Electronics	TV	55000	Completed	55000	55000	U101	North	High
P102	HP Laptop	Electronics	Laptop	72000	Pending	72000	72000	U102	West	Medium
P103	Rice Bag	Groceries	Rice	2500	Completed	2500	2500	U103	South	Low
P104	Office Chair	Furniture	Chair	8500	Completed	8500	8500	U104	East	Medium

```
[11] from pyspark.sql.functions import col
✓ 0s

[12] filtered_df = parquet_df.filter(col("user_id").isNotNull())
✓ 1s
    filtered_df.show()
```

product_id	product_name	category	old_name	price	status	amount	base_price	user_id	region	priority
P101	Samsung TV	Electronics	TV	55000	Completed	55000	55000	U101	North	High
P102	HP Laptop	Electronics	Laptop	72000	Pending	72000	72000	U102	West	Medium
P103	Rice Bag	Groceries	Rice	2500	Completed	2500	2500	U103	South	Low
P104	Office Chair	Furniture	Chair	8500	Completed	8500	8500	U104	East	Medium
P105	Mixer Grinder	Electronics	Mixer	4200	Cancelled	4200	4200	U105	North	High
P106	Dining Table	Furniture	Table	25000	Completed	25000	25000	U106	West	Low
P107	Jeans	Clothing	Jeans	1800	Pending	1800	1800	U107	South	Medium
P109	Milk	Groceries	Milk	60	Completed	60	60	U109	East	Low
P110	Air Conditioner	Electronics	AC	48000	Completed	48000	48000	U110	West	High
P111	Bookshelf	Furniture	Shelf	12000	Pending	12000	12000	U111	North	Medium
P112	T-Shirt	Clothing	Tee	900	Completed	900	900	U112	South	Low
P113	Washing Machine	Electronics	Washer	38000	Completed	38000	38000	U113	North	High
P114	Sofa	Furniture	Sofa	45000	Cancelled	45000	45000	U114	West	High
P115	Headphones	Electronics	Headset	3000	Completed	3000	3000	U115	North	Medium
P116	Cooking Oil	Groceries	Oil	2200	Completed	2200	2200	U116	East	High
P117	Refrigerator	Electronics	Fridge	65000	Pending	65000	65000	U117	South	High
P118	Curtains	Home Decor	Curtain	1500	Completed	1500	1500	U118	North	Low
P119	Microwave Oven	Electronics	Microwave	11000	Completed	11000	11000	U119	West	Medium
P120	Study Lamp	Electronics	Lamp	1200	Completed	1200	1200	U120	North	High

```
[20] filtered_df.write \
✓ 0s
    .mode("overwrite") \
    .option("header", True) \
    .csv("output_csv")
```

Q13: In Spark Architecture, what is the difference between **Client Mode** and **Cluster Mode**?

Ans:-

In **Client Mode**, the Driver program runs on the user's local machine, while the executors run on the cluster. This mode is useful for development, testing, and debugging because the user can directly monitor the application.

In **Cluster Mode**, the Driver runs inside the cluster along with the executors. The cluster manager is responsible for managing both the Driver and the executors. This mode is more suitable for production environments because the application continues running even if the client machine disconnects.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

product_id	product_name	category	old_name	price	status	amount	base_price	user_id	region	priority
P101	Samsung TV	Electronics	TV	55000	Completed	55000	55000	U101	North	High
P105	Mixer Grinder	Electronics	Mixer	4200	Cancelled	4200	4200	U105	North	High
P108	Smartphone	Electronics	Phone	35000	Completed	35000	35000	NULL	North	High
P110	Air Conditioner	Electronics	AC	48000	Completed	48000	48000	U110	West	High
P111	Bookshelf	Furniture	Shelf	12000	Pending	12000	12000	U111	North	Medium
P113	Washing Machine	Electronics	Washer	NULL	Completed	38000	38000	U113	North	High
P114	Sofa	Furniture	Sofa	45000	Cancelled	45000	45000	U114	West	High
P115	Headphones	Electronics	Headset	3000	Completed	3000	3000	U115	North	Medium
P116	Cooking Oil	Groceries	Oil	2200	Completed	2200	2200	U116	East	High
P117	Refrigerator	Electronics	Fridge	65000	Pending	65000	65000	U117	South	High
P118	Curtains	Home Decor	Curtain	1500	Completed	1500	1500	U118	North	Low
P120	Study Lamp	Electronics	Lamp	1200	Completed	1200	1200	U120	North	High

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Ans:-

The `.show(5)` method displays only the first five rows of the dataset, making it fast and memory-efficient. In contrast, `.collect()` retrieves the entire dataset to the Driver program. On a multi-terabyte dataset, this can consume a large amount of memory and may even cause the Driver to run out of memory or crash. Therefore, `.show(5)` is a much safer option for exploring large datasets.